

Name-Krish Satpal
Roll no.28

Experiment No. 5

□ **Aim:** Hands on Solidity Programming Assignments for creating Smart Contracts

□ **Theory:**

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:

- o public: available both inside and outside the contract.
- o private: only accessible within the same contract.
- o internal: accessible within the contract and its child contracts.
- o external: can be called only by external accounts or other contract

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

? Implementation:

- Tutorial no. 1 – Compile the code

The screenshot displays the Remix IDE interface. At the top, a browser tab shows 'Learneth Solidity Beginner Course'. The main workspace is divided into three panels:

- Debugger (Left):** Contains a 'Start debugging' section with instructions to select a past transaction or paste a hash. Below this is a 'Transactions recorder 1' panel showing a 'Counter' contract call at address 0x19e...9f9e8, deployed 20s ago. A 'constructor' transaction is also listed.
- Compiler (Middle):** Shows the Solidity code for 'introduction.sol'. The code includes functions for getting, incrementing, and decrementing a count. The 'Compiled' status is indicated at the top of the panel.
- Console (Bottom):** Displays a log entry: '[vm] from: 0x583...edd64 to: Counter.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0x4ef...0e405'. A 'Debug' button is visible next to the log.

The top of the interface includes a 'Link' button, 'Sign In', 'See Plans', and 'Get AI Credits' options. The bottom right corner features an 'AI copilot' toggle.

- Tutorial no. 2

The screenshot shows the Remix IDE interface with the debugger open. The top bar includes the Remix logo, a search bar, and navigation links like 'Link', 'Sign In', 'See Plans', and 'Get AI Credits'. The left sidebar contains various tool icons. The main area is divided into three panels:

- Debugger Panel (Left):** Displays transaction details for a successful transaction (Block: 4, Timestamp: 8/31/2026, 11:17:11 AM). It also shows a call trace for 'Contract Creation' and a 'State & Locals' view.
- Code Editor (Right):** Shows the Solidity code for the 'HelloWorld' contract. The code includes a SPDX license header, a pragma statement for Solidity 0.8.3, and a contract definition with a public string variable 'greet'.
- Execution Trace (Bottom Right):** A section for viewing the execution trace of the selected call from the call trace.

- Tutorial no. 3

The screenshot shows the Remix IDE interface with the debugger open. The top bar includes the Remix logo, a search bar, and navigation links like 'Link', 'Sign In', 'See Plans', and 'Get AI Credits'. The left sidebar contains various tool icons. The main area is divided into three panels:

- Debugger Panel (Left):** Displays transaction details for a successful transaction (Block: 5, Timestamp: 8/31/2026, 11:21:25 AM). It also shows a call trace for 'Contract Creation' and a 'State & Locals' view.
- Code Editor (Right):** Shows the Solidity code for the 'Primitives' contract. The code includes a SPDX license header, a pragma statement for Solidity 0.8.3, and a contract definition with a public boolean variable 'boo' and several unsigned integer variables.
- Execution Trace (Bottom Right):** A section for viewing the execution trace of the selected call from the call trace.

- Tutorial no. 4

The screenshot shows the Remix IDE interface for Tutorial no. 4. The debugger is active, displaying transaction details for a successful transaction. The call trace shows the 'CREATE' function being executed. The state and locals are visible, showing an empty 'locals' object and a 'state' object. The code editor displays the Solidity code for the 'Variables' contract, which includes a 'doSomething' function and an 'add' function. The interface also includes a sidebar with a file explorer, a central console area, and a right-hand pane for the Solidity code editor.

- Tutorial no. 5

The screenshot shows the Remix IDE interface for Tutorial no. 5. The debugger is active, displaying transaction details for a successful transaction. The call trace shows the 'CREATE' function being executed. The state and locals are visible, showing an empty 'locals' object and a 'state' object. The code editor displays the Solidity code for the 'ViewAndPure' contract, which includes an 'addToX' function and an 'add' function. The interface also includes a sidebar with a file explorer, a central console area, and a right-hand pane for the Solidity code editor.

- Tutorial no. 6

REMIX 2.5.5 Learneth Solidity Beginner Course

DEBUGGER

Qxd324ed4e620544587d3c162cb8331110fca13f5a42edda9e2cc0955e1

Use generated sources(Solidity >= v0.7.2)

Transaction Details

Status: Success Tx Fee: 9034920000000000 Wei

Block: 10 Tx Type: Type 0

Timestamp: 8/31/2026, 11:26:05 AM Gas Price: 20000000000 Wei

From: 0x5838...ddc4 Gas Used: 451746

To: 0xddda...482d Tx Index: 0

Call Trace (Step: 12 / 40)

SENDER 0x5838Da6a701c56854dCfc803Fc875f56beddC4

CREATE Contract Creation

State & Locals

Stack & Memory

locals: {}

state: { ... }

Execution trace

Select a call from Call Trace to view execution details

contract FunctionModifier {

// We will use these variables to demonstrate how to use

// modifiers.

address public owner;

uint public x = 10;

bool public locked;

constructor() {

// Set the transaction sender as the owner of the contract;

owner = msg.sender;

}

// Modifier to check that the caller is the owner of

// the contract.

modifier onlyOwner() {

require(msg.sender == owner, "Not owner");

}

- Tutorial no. 7

REMIX 2.5.5 Learneth Solidity Beginner Course

DEBUGGER

0xf5cc3ad0b45a113b2005b8d72ea600965b44046bad36860266ae575

Use generated sources(Solidity >= v0.7.2)

Transaction Details

Status: Success Tx Fee: 7931020000000000 Wei

Block: 12 Tx Type: Type 0

Timestamp: 8/31/2026, 11:29:24 AM Gas Price: 20000000000 Wei

From: 0x5838...ddc4 Gas Used: 396551

To: 0xb27a...7c2c Tx Index: 0

Call Trace (Step: 0 / 16)

SENDER 0x5838Da6a701c56854dCfc803Fc875f56beddC4

CREATE Contract Creation

State & Locals

Stack & Memory

locals: {}

state: { ... }

Execution trace

Select a call from Call Trace to view execution details

contract Function {

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.3;

function returnMany() public pure returns (uint, bool, uint) {

return (1, true, 2);

}

- Tutorial no. 8

The screenshot shows the Remix IDE interface. On the left, the 'DEBUGGER' panel is active, displaying 'Transaction Details' and 'Call Trace'. The 'Transaction Details' section shows a successful transaction with a fee of 679764000000000 Wei. The 'Call Trace' section shows a 'CREATE' operation for 'Contract Creation'. The main editor displays a Solidity contract named 'Base' with a 'privateFunc' and a 'testPrivateFunc' function. A 'debugging' tooltip is visible over the code, showing the transaction hash: 0x4ad8a167476134ba819e5f4995a5f8f240fc1d90fb366beaa551183d5f366beaa551183d53bdfef9f. The bottom status bar includes a 'Scam Alert' and 'Initialize as git repo' button.

- Tutorial no. 9

The screenshot shows the Remix IDE interface. On the left, the 'DEBUGGER' panel is active, displaying 'Transaction Details' and 'Call Trace'. The 'Transaction Details' section shows a successful transaction with a fee of 287602000000000 Wei. The 'Call Trace' section shows a 'CREATE' operation for 'Contract Creation'. The main editor displays a Solidity contract named 'IfElse' with a 'function foo' and a 'function ternary' function. The bottom status bar includes a 'Scam Alert' and 'Initialize as git repo' button.

- Tutorial no. 10

The screenshot shows the REMIX IDE interface for Tutorial no. 10. The left sidebar contains the 'DEBUGGER' panel with the following details:

- Transaction Details:**
 - Status: Success
 - Block: 15
 - Timestamp: 8/31/2026, 11:33:14 AM
 - From: 0x5B38...ddc4
 - To: 0xd8d3...5692
 - Tx Fee: 2342380000000000 Wei
 - Tx Type: Type 0
 - Gas Price: 20000000000 Wei
 - Gas Used: 117119
 - Tx Index: 0
- Call Trace (Step: 0 / 16):**
 - SENDER: 0x5B38Da6a701c56854dCfc803Fc875f56beddC4
 - CREATE: Contract Creation
- State & Locals:**
 - locals: {}
 - state: {}

The main editor shows a Solidity contract named 'Loop' with the following code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5     function loop() public {
6         for (uint i = 0; i < 10; i++) {
7             if (i == 3) {
8                 // skip to next iteration with continue
9                 continue;
10            }
11            if (i == 5) {
12                // Exit loop with break
13                break;
14            }
15        }
16    }
17 }
```

The right sidebar shows the 'Execution trace' panel with the text: 'Select a call from Call Trace to view execution details'.

- Tutorial no. 11

The screenshot shows the REMIX IDE interface for Tutorial no. 11. The left sidebar contains the 'DEBUGGER' panel with the following details:

- Transaction Details:**
 - Status: Success
 - Block: 16
 - Timestamp: 8/31/2026, 11:34:17 AM
 - From: 0x5B38...ddc4
 - To: 0xd4fc...cbee
 - Tx Fee: 8661800000000000 Wei
 - Tx Type: Type 0
 - Gas Price: 20000000000 Wei
 - Gas Used: 433090
 - Tx Index: 0
- Call Trace (Step: 0 / 421):**
 - SENDER: 0x5B38Da6a701c56854dCfc803Fc875f56beddC4
 - CREATE: Contract Creation
- State & Locals:**
 - locals: {}
 - state: {}

The main editor shows a Solidity contract named 'Array' with the following code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     // Fixed sized array, all elements initialize to 0
9     uint[10] public myFixedSizeArr;
10
11     function get(uint i) public view returns (uint) {
12         return arr[i];
13     }
14
15     // Solidity can return the entire array.
16     // But this function should be avoided for
```

The right sidebar shows the 'Execution trace' panel with the text: 'Select a call from Call Trace to view execution details'.

- Tutorial no. 12

REMIX 2.5.5 Learneth Solidity Beginner Course

DEBUGGER

0x223a2d9346f40c8c4226e0c6e43f15b5c07241124a02ec04cd4c45eb

Use generated sources(Solidity >= v0.7.2)

Transaction Details

Status: Success Tx Fee: 451560000000000 Wei

Block: 17 Tx Type: Type 0

Timestamp: 8/31/2026, 11:35:19 AM Gas Price: 20000000000 Wei

From: 0x5838...ddc4 Gas Used: 225780

To: 0x5FD6...c880 Tx Index: 0

Call Trace (Step: 0 / 16)

SENDER 0x5838Da6a701c568545dcfcb03fcb875f56beddC4

CREATE Contract Creation

State & Locals

Stack & Memory

locals: {}

state: { ... }

Debug with RemixAI

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public myMap;
7
8     function get(address _addr) public view returns (uint) {
9         // Mapping always returns a value.
10        // If the value was never set, it will return the default value.
11        return myMap[_addr];
12    }
13
14    function set(address _addr, uint _i) public {
15        // Update the value at this address
16        myMap[_addr] = _i;
17    }
18 }
```

debugging 0x223a2d9346f40c8c4226e0c6e43f15b5c07241124a02ec04cd4c45eb759b663

Previous Breakpoint Step Backward Step Back Step Into Step Forward Next Breakpoint

Execution trace

Select a call from Call Trace to view execution details

Scam Alert Initialize as git repo Did you know? To learn new contract patterns and prototype, you can activate and try the cookbook plugin! RemixAI Copilot (enabled)

- Tutorial no. 13

REMIX 2.5.5 Learneth Solidity Beginner Course

DEBUGGER

0x442eb148e1301652889d5eba3568149e19ac319a43764caa588ec239

Use generated sources(Solidity >= v0.7.2)

Transaction Details

Status: Success Tx Fee: 1299610000000000 Wei

Block: 18 Tx Type: Type 0

Timestamp: 8/31/2026, 11:37:20 AM Gas Price: 20000000000 Wei

From: 0x5838...ddc4 Gas Used: 649805

To: 0x7b96...6ac6 Tx Index: 0

Call Trace (Step: 0 / 16)

SENDER 0x5838Da6a701c568545dcfcb03fcb875f56beddC4

CREATE Contract Creation

State & Locals

Stack & Memory

locals: {}

state: { ... }

Debug with RemixAI

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Todos {
5     struct Todo {
6         string text;
7         bool completed;
8     }
9
10    // An array of 'Todo' structs
11    Todo[] public todos;
12
13    function create(string memory _text) public {
14        // 3 ways to initialize a struct
15        // - calling it like a function
16        todos.push(Todo(_text, false));
17    }
18 }
```

Previous Breakpoint Step Backward Step Back Step Into Step Forward Next Breakpoint

Execution trace

Select a call from Call Trace to view execution details

Scam Alert Initialize as git repo Did you know? To learn new contract patterns and prototype, you can activate and try the cookbook plugin! RemixAI Copilot (enabled)

- Tutorial no. 14

The screenshot shows the Remix IDE interface for Tutorial no. 14. The left sidebar contains the 'DEBUGGER' panel with a search bar, transaction details, call trace, and state/locals. The main editor shows a Solidity contract with an Enum for shipping status. The right sidebar shows the 'Execution trace' section.

Transaction Details:

Status: Success	Tx Fee: 3711740000000000 Wei
Block: 19	Tx Type: Type 0
Timestamp: 8/31/2026, 11:37:47 AM	Gas Price: 20000000000 Wei
From: 0x5838...ddc4	Gas Used: 185587
To: 0x3328...4860	Tx Index: 0

Call Trace (Step: 0 / 16):

- SENDER: 0x5838De6a701c568545dcfc803Fc875f56beddc4
- CREATE Contract Creation

State & Locals:

```
{
  locals: {}
  state: { ...
}
```

Solidity Code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Enum representing shipping status
6     enum Status {
7         Pending,
8         Shipped,
9         Accepted,
10        Rejected,
11        Canceled
12    }
13
14    // Default value is the first element listed in
15    // definition of the type, in this case "Pending"
16    Status public status;
```

- Tutorial no. 15

The screenshot shows the Remix IDE interface for Tutorial no. 15. The left sidebar contains the 'DEBUGGER' panel with a search bar, transaction details, call trace, and state/locals. The main editor shows a Solidity contract with a struct and a mapping. The right sidebar shows the 'Execution trace' section.

Transaction Details:

Status: Success	Tx Fee: 6775800000000000 Wei
Block: 20	Tx Type: Type 0
Timestamp: 8/31/2026, 11:38:43 AM	Gas Price: 20000000000 Wei
From: 0x5838...ddc4	Gas Used: 338790
To: 0x5e17...Efff5	Tx Index: 0

Call Trace (Step: 0 / 16):

- SENDER: 0x5838De6a701c568545dcfc803Fc875f56beddc4
- CREATE Contract Creation

State & Locals:

```
{
  locals: {}
  state: { ...
}
```

Solidity Code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7     struct MyStruct {
8         uint foo;
9     }
10    mapping(uint => MyStruct) myStructs;
11
12    function f() public {
13        // call _f with state variables
14        _f(arr, map, myStructs[1]);
15    }
16    // get a struct from a mapping
```

- Tutorial no. 16

REMIX 2.5.5 Learneth Solidity Beginner Course

DEBUGGER

Q 0xea27bdda090f5e478dcdcea2d461d3ec88387ab50a197b46444a40c7c

Transaction Details

Status: Success Tx Fee: 4584760000000000 Wei

Block: 21 Tx Type: Type 0

Timestamp: 8/31/2026, 11:39:37 AM Gas Price: 20000000000 Wei

From: 0xc5838...ddc4 Gas Used: 229238

To: 0xe289...157a Tx Index: 0

Call Trace (Step: 0 / 68)

SENDER 0x5838Da6a701c568545dCfc803Fc875f56beddC4

CREATE Contract Creation

State & Locals Stack & Memory

locals: {}

state: { ... }

debugging 0xea27bdda090f5e478dcdcea2d461d3ec88387ab50a197b46444a40c7c3ea4419

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12 }
```

- Tutorial no. 17

REMIX 2.5.5 Learneth Solidity Beginner Course

DEBUGGER

Q 0xa1fc0afd3cde6fcc9b089cef87215072828dcf68240ba53b930e4c0857

Transaction Details

Status: Success Tx Fee: 2568060000000000 Wei

Block: 22 Tx Type: Type 0

Timestamp: 8/31/2026, 11:40:10 AM Gas Price: 20000000000 Wei

From: 0xc5838...ddc4 Gas Used: 128403

To: 0x1c91...b4b0 Tx Index: 0

Call Trace (Step: 0 / 19)

SENDER 0x5838Da6a701c568545dCfc803Fc875f56beddC4

CREATE Contract Creation

State & Locals Stack & Memory

locals: {}

state: { ... }

debugging 0xa1fc0afd3cde6fcc9b089cef87215072828dcf68240ba53b930e4c0857b6e26

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6
7     // Using up all of the gas that you send causes your transaction to fail.
8     // State changes are undone.
9     // Gas spent are not refunded.
10    function forever() public {
11        infinite gas
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

- Tutorial no. 18

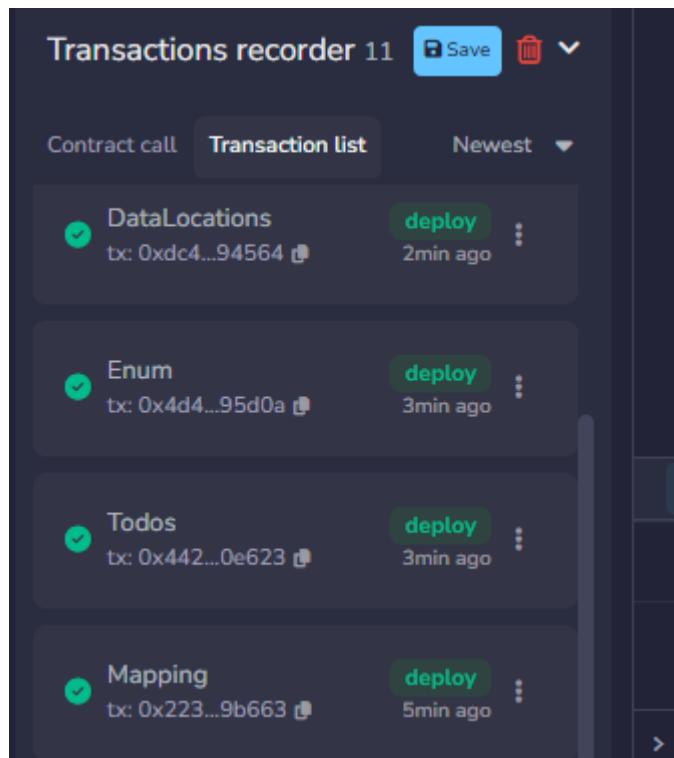
The screenshot shows the Remix IDE interface. The top bar includes the Remix logo, version 2.5.5, and a 'Learneth Solidity Beginner Course' link. The left sidebar has a 'DEBUGGER' panel with a search bar and a 'Stop' button. Below the search bar, there's a checkbox for 'Use generated sources(Solidity >= v0.7.2)'. The 'Transaction Details' section shows a successful transaction with a Tx Fee of 182990000000000 Wei, Block 23, and a timestamp of 8/31/2026, 11:48:58 AM. The 'Call Trace (Step: 0 / 16)' section shows a 'SENDER' transaction with a 'CREATE' operation. The 'State & Locals' section shows an empty state and locals. The main editor displays a Solidity contract named 'ReceiveEther' with the following code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ReceiveEther {
5     // PUSH costs 3 gas - this line costs 85 gas - 49010 gas left
6     // which function is called, fallback() or receive()?
7     //
8     send Ether
9     //
10    msg.data is empty?
11    //
12    yes no
13    //
14    receive() exists? fallback()
15    //
16    yes no
```

- Tutorial 19

The screenshot shows the Remix IDE interface. The top bar includes the Remix logo, version 2.5.5, and a 'Learneth Solidity Beginner Course' link. The left sidebar has a 'DEBUGGER' panel with a search bar and a 'Stop' button. Below the search bar, there's a checkbox for 'Use generated sources(Solidity >= v0.7.2)'. The 'Transaction Details' section shows a successful transaction with a Tx Fee of 3711740000000000 Wei, Block 19, and a timestamp of 8/31/2026, 11:37:47 AM. The 'Call Trace (Step: 0 / 16)' section shows a 'SENDER' transaction with a 'CREATE' operation. The 'State & Locals' section shows an empty state and locals. The main editor displays a Solidity contract named 'Enum' with the following code:

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Enum representing shipping status
6     enum Status {
7         Pending,
8         Shipped,
9         Accepted,
10        Rejected,
11        Canceled
12    }
13
14    // Default value is the first element listed in
15    // definition of the type, in this case "Pending"
16    Status public status;
```



□ **Conclusion:**

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.